Chapter 17: Deploying ML Pipelines

<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to design end-to-end ML pipeline architectures with separate training and inference code paths
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to implement ETL processes that extract, transform, and load data from operational systems into analytics-ready formats
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to serialize trained models using joblib and implement model versioning with training metadata
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to build inference pipelines that load saved models and generate predictions on new data in production environments
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to implement scheduled retraining workflows that maintain model currency over time

17.1Introduction

What Deployment Really Means

In many courses, “deployment” is presented as something that happens only in the cloud—through external APIs, managed platforms, and specialized

MLOps tools. Those approaches matter, but they are not where most real-world machine learning work begins.

In practice, the first version of a deployed model is often *embedded directly inside an application*. A team trains a model, saves it as a file, and loads it inside the same codebase that powers the product. The application reads fresh data from a database, runs the model to produce predictions, and logs results for monitoring and improvement. In practice, this often includes a lightweight analytical copy of operational data created through scheduled ETL, even when everything runs on the same machine.

This chapter teaches that foundational deployment pattern: an *end-to-end machine learning pipeline* that lives in your application environment. You will not use AWS, Azure, or external model-serving APIs here. Instead, you will build a complete pipeline that you can run on your own machine or in a controlled classroom environment.

This approach is intentionally simple. It is not the most scalable or the most sophisticated architecture—but it is *realistic*, *teachable*, and *extremely common* in early-stage products, internal analytics tools, and small-team deployments.

What This Chapter Builds

By the end of this chapter, you will be able to run a complete ML pipeline using a small set of Python scripts that do the following:

- Load data directly from a live operational database (the same database used by the application).
- Apply automated cleaning and feature engineering using reusable pipeline functions.

- Train a model and evaluate it using a standard train/test workflow.
- Save the trained model to disk as a versioned file (for example, a *.sav* artifact).
- Schedule retraining to run automatically (for example, nightly or weekly).
- Load the saved model inside an application-like script and run predictions on new records.

This is what “deployment” often looks like before a team invests in data warehouses, orchestration platforms, and enterprise MLOps systems. Understanding this foundation will make advanced tools easier to learn later because you will know what those tools are automating.

Key Mental Model: Deployment Is About Reliability

A deployed model is not defined by where it runs. It is defined by whether it can run *reliably* and *repeatably* in a real process that other people or systems depend on.

This chapter emphasizes practical reliability principles that apply in every environment—from a single Python script to enterprise cloud platforms:

- **Repeatability:** the same pipeline produces the same outputs when run on the same inputs.
- **Traceability:** you can identify which code, data, and model version produced a prediction.
- **Separation of concerns:** training code, inference code, and data access code are organized clearly.

- **Safe failure:** the system fails gracefully (with clear logging) instead of silently producing unreliable outputs.

Why We Start Here (Before Cloud Platforms)

Cloud platforms like Azure ML Studio and managed MLOps pipelines are valuable, but they can hide the basic moving parts. In this chapter, you will build the moving parts yourself so you understand what is happening at each step.

Later, when you use managed tools, you will recognize the same pipeline components—data loading, cleaning, training, evaluation, model registration, scheduling, and monitoring—just wrapped in a larger system.

Hands-On Preview: The Simplest "Deployed Model"

To preview what deployment means in this chapter, here is the smallest possible example. We train a model, save it as a file, and load it later to make predictions. You will build a more complete version of this workflow throughout the chapter.

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification
import joblib

# 1. Train a small model (toy example)
X, y = make_classification(n_samples=500, n_features=8, random_state=27)
model = LogisticRegression(max_iter=500)
model.fit(X, y)

# 2. Save the model artifact to disk
joblib.dump(model, &quot;model.sav&quot;)

# 3. Load the model artifact later (simulating an application restart)
loaded = joblib.load(&quot;model.sav&quot;)
```

```
# 4. Run inference on new data
x_new = np.random.rand(1, 8)
pred = loaded.predict(x_new)
proba = loaded.predict_proba(x_new)

print(&quot;Prediction:&quot;, int(pred[0]))
print(&quot;Probabilities:&quot;, proba[0])


# Output:
# Prediction: 1
# Probabilities: [0.05629117 0.94370883]
```

This example demonstrates a core deployment truth: a trained model is just an artifact that can be persisted and reused. The rest of deployment work is about building a stable pipeline around that artifact, ensuring that the data and transformations used during training match what will happen in production.

In the next section, you will see the full embedded deployment architecture and how all parts of the pipeline connect: application, operational database, training script, model artifact, retraining schedule, and inference process.

17.2Deployment Architecture and Pipeline Design

A Simple but Realistic ML Deployment Architecture

In this chapter, “deployment” means connecting your model to a real application workflow: data flows into a live database, a scheduled training script rebuilds the model periodically, the model is saved to disk, and the application loads that saved model to make predictions on new records.

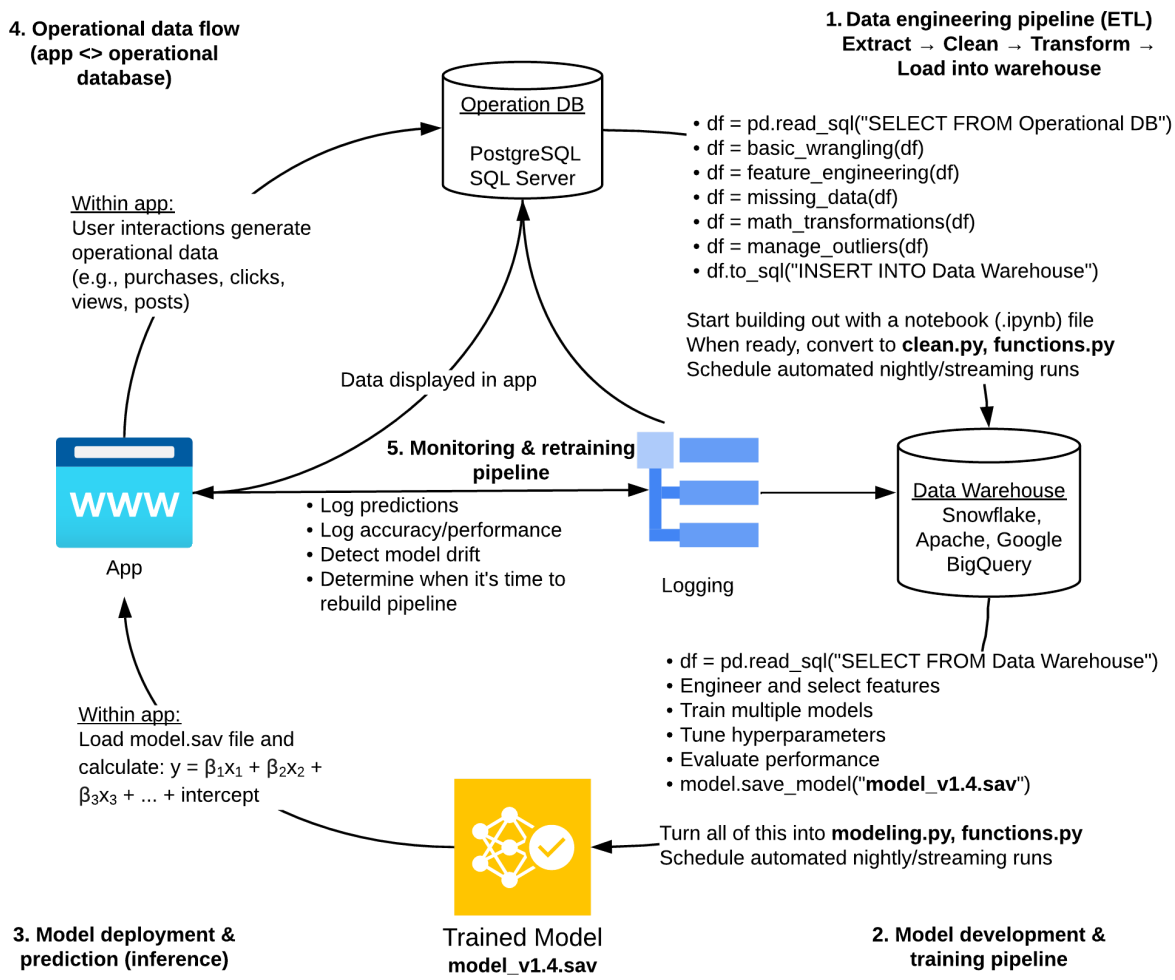
This design is intentionally simple: the training process reads directly from the same operational database used by the application. In production, many organizations insert an analytics layer (a warehouse or lakehouse) between the operational database and modeling, but this streamlined version is

realistic for small teams and many early-stage products. *This approach still allows for lightweight analytical copies of the data created via scheduled ETL without introducing full-scale infrastructure.*

Core Components

- **Application**: collects inputs, writes new records, and requests predictions during user workflows.
- **Operational database**: the live system of record (for example, PostgreSQL, MySQL, SQL Server) storing application data.
- **Periodic training script**: a Python program that queries the database, cleans data, trains a model, evaluates it, and saves artifacts.
- **Saved model file**: a serialized artifact (for example, a *.sav* file) that the application can load for inference.
- **Inference code path**: application logic that loads the latest model and produces predictions for new inputs.

Architecture Diagram



Pipeline Stages

An end-to-end deployment pipeline is simply a structured way to turn the CRISP-DM process into executable code. Every stage you learned earlier—understanding data, preparing it, modeling, and evaluation—still exists. The difference is that these steps now run automatically and repeatedly. Deployment is not a new discipline. It is *CRISP-DM operationalized*.

A well-designed pipeline breaks the workflow into clear, testable stages. Each stage should be implemented as a function or module that can be reused, logged, and debugged independently. In practice, early stages are

often executed as a lightweight ETL process that prepares analytical-ready data for modeling.

- **Data ingestion**: Connect to the operational database, query relevant records, and load them into a working DataFrame. This step defines the snapshot of data used for training.
- **Automated cleaning**: Apply the same reusable cleaning functions developed earlier in the course (wrangling, dates, bins, missing data, outliers). No dataset-specific logic should appear beyond this stage.
- **Feature engineering**: Transform cleaned data into model-ready features, including encoding, scaling, and derived variables. These transformations must be identical during training and inference, ideally enforced through shared pipeline code.
- **Model training**: Fit the selected algorithm using the engineered features. This step mirrors traditional supervised learning and should include clear configuration of hyperparameters.
- **Evaluation**: Measure performance using a validation or holdout set. Metrics should be logged and compared over time to detect degradation or improvement.
- **Model serialization**: Save the trained model (and any required preprocessing objects) to disk in a standardized format such as *.sav*. This artifact is what the application will load for inference.

Why This Structure Matters

Separating the pipeline into stages enforces discipline. Each step has a single responsibility, which makes the system easier to test, modify, and explain.

Most deployment failures are not modeling failures—they are pipeline failures. Clear boundaries and reusable components reduce the risk of data leakage, inconsistent preprocessing, and silent behavior changes.

CRISP-DM Turned Into Code

If this structure feels familiar, it should. The pipeline maps directly to CRISP-DM:

- Business understanding → defining the prediction task and evaluation criteria
- Data understanding → ingestion and basic validation
- Data preparation → automated cleaning and feature engineering
- Modeling → training and tuning
- Evaluation → metric computation and comparison
- Deployment → serialization and integration with the application

The core insight is simple but powerful: deployment does not replace analytics fundamentals—it forces you to apply them consistently, every time the pipeline runs.

Where Warehouses and Lakehouses Fit Later

At larger scale, teams often separate operational and analytics workloads by copying data into a warehouse or lakehouse (for example, Snowflake or a Spark-based platform). That separation improves performance, governance, reproducibility, and historical tracking, but the core pipeline steps you practice here still apply.

17.3ETL

In real-world systems, machine learning models are rarely trained directly on live operational databases. Instead, data is extracted, transformed, and loaded (ETL) into a separate analytical store that is optimized for modeling.

In this chapter, we simulate that production pattern using two SQLite databases:

This download can be found online.

- *shop.db* — the live operational database used by the application
- *warehouse.db* — a denormalized analytical database used for modeling

This separation reinforces an important deployment principle: models should train on stable, well-structured data, not directly on transactional tables. The ETL process can be run repeatedly on a schedule, producing the same analytical output given the same source data.

What This ETL Script Does

The ETL process in this section performs four core steps:

- Extract relevant tables from the operational database
- Join and denormalize the data into a single modeling table
- Perform light, repeatable cleaning and feature construction
- Load the result into a separate SQLite database

All dataset-specific logic remains here. Downstream modeling code will assume that the data is already clean, consistent, and modeling-ready.

Extract and Join Operational Data

We begin by connecting to the operational database and loading all five tables into pandas DataFrames. In production systems, you would typically query only the columns and date ranges you need. For this chapter, we load everything so you can explore the data interactively and understand the full schema before deciding which columns to keep.

```
import sqlite3
import pandas as pd

# Connect to operational database
conn = sqlite3.connect(&quot;shop.db&quot;)

# Load core tables
orders = pd.read_sql(&quot;SELECT * FROM orders&quot;, conn)
customers = pd.read_sql(&quot;SELECT * FROM customers&quot;, conn)
order_items = pd.read_sql(&quot;SELECT * FROM order_items&quot;, conn)
products = pd.read_sql(&quot;SELECT * FROM products&quot;, conn)
shipments = pd.read_sql(&quot;SELECT * FROM shipments&quot;, conn)
conn.close()

print(orders.shape, customers.shape, order_items.shape, products.shape,
shipments.shape)


# Output:
# (5000, 17) (250, 12) (15022, 6) (100, 7) (5000, 9)
```

Each tuple shows (rows, columns) for the five tables. Notice how normalized the data is: 5,000 orders are spread across five separate tables, each optimized for transactional processing rather than analytics. The *order_items* table has over 15,000 rows because a single order can contain multiple line items. Our goal in the next steps is to collapse all of this into *one row per order* with features suitable for modeling.

Denormalize to One Row per Order

The *order_items* table stores one row per product in each order. To use this data for modeling, we need to aggregate it so that each order gets a single row of summary statistics. The *groupby/agg* pattern below computes four features per order: total quantity ordered, number of distinct products, average unit price across line items, and total dollar value across all line items.

```
# Aggregate order items per order
order_item_features = (
  order_items
    .groupby(&quot;order_id&quot;)
    .agg(
      num_items=(&quot;quantity&quot;, &quot;sum&quot;),
      num_distinct_products=(&quot;product_id&quot;, &quot;nunique&quot;),
      avg_unit_price=(&quot;unit_price&quot;, &quot;mean&quot;),
      total_line_value=(&quot;line_total&quot;, &quot;sum&quot;)
    )
    .reset_index()
)

order_item_features.head()
```

```
# Output:
#    order_id  num_items  num_distinct_products  avg_unit_price  total_line_value
# 0         1          9                      5          69.242            662.95
# 1         2          7                      5         133.300            862.92
# 2         3          5                      3         140.850            796.09
# 3         4          1                      1         137.600            137.60
# 4         5          1                      1          17.070             17.07
```

Each row now represents one order. Notice how order 1 originally had multiple line items (9 total items across 5 distinct products) that are now summarized into a single row. Next, we join this aggregated data with the orders, customers, and shipments tables to create a single denormalized table that has everything we need for modeling.

```
# Join everything into one modeling table
df = (
  orders
    .merge(customers, on=&quot;customer_id&quot;, how=&quot;left&quot;)
```

```
        .merge(order_item_features, on=&quot;order_id&quot;, how=&quot;left&quot;)
        .merge(shipments[[&quot;order_id&quot;, &quot;late_delivery&quot;]],
on=&quot;order_id&quot;, how=&quot;left&quot;)
    )

    df.head()


    # Output (5000 rows x 33 columns — selected columns shown):
    #    order_id  customer_id  ...  num_items  total_line_value  late_delivery
    # 0         1            1  ...          9            662.95              1
    # 1         2            1  ...          7            862.92              1
    # 2         3            1  ...          5            796.09              1
```

The result is a wide table with 33 columns—order details, customer demographics, aggregated item statistics, and the shipment outcome. We use *left joins* throughout so that every order is preserved even if a related record is missing. Notice that we only take *late_delivery* from the shipments table (not carrier, ship_datetime, or other shipment details) because those fields describe what happened *after* the order was placed and would leak future information into the model.

Clean and Prepare with Reusable Functions

Earlier in this course, you built a *functions.py* file containing reusable >functions.py into the same folder as this notebook (or your ETL script) and import the functions you need.

```
    from functions import missing_data_diagnostics, missing_data_clean, manage_dates

    # Examine data quality after the join
    diagnostics = missing_data_diagnostics(df)


    # Output:
    # === Missing data diagnostics ===
    # Threshold: drop if proportion missing &gt; 0.9
    # Columns that would be dropped (0): none
    # Rows that would be dropped: 0
    #
    # Per-column summary (columns with missing):
    #   promo_code: missing=3739 (74.78%), suggested mechanism=MAR/MNAR
    # (MAR vs MNAR cannot be distinguished from data; 'MAR/MNAR' means
```

```
#  missingness is associated with observed variables.)
```

The diagnostics report reveals that *promo_code* is missing in about 75% of rows—which makes sense, because most orders do not use a promotional code. The function flags this as MAR/MNAR because the presence of a promo code is correlated with other order characteristics (customers who use promos tend to have different spending patterns). Fortunately, our modeling table uses the binary *promo_used* flag rather than the code text itself, so this missingness does not affect the features we actually model with. In a production pipeline, this same report would feed into monitoring dashboards or trigger alerts when data quality degrades beyond acceptable thresholds.

```
# Clean missing data (drops columns/rows above threshold, imputes the rest)
df = missing_data_clean(df, missing_thresh=0.9, diagnostics=False)
print(f&quot;After cleaning: {df.shape}&quot;)


# Output:
# After cleaning: (5000, 33)
```

The cleaning function drops any column or row where more than 90% of values are missing, then imputes remaining numeric gaps with the median and categorical gaps with the mode. Here the dataset retains all 5,000 rows and 33 columns because no column exceeds the 90% threshold (*promo_code* at 75% is below it and gets imputed instead). Running this as a standard pipeline step ensures that the analytical database always receives clean data, even if future source data contains unexpected gaps.

Feature Engineering for Late Delivery

With the data cleaned, we now engineer features that plausibly influence delivery timing. The *manage_dates* function from your functions.py handles

date parsing and temporal decomposition automatically—it converts a date column to datetime format and extracts day, month, year, weekday, and hour as new columns. We then add two domain-specific features that require custom logic: customer age (a derived relationship between two date columns that *manage_dates* cannot compute on its own) and customer order count (a groupby aggregation across the dataset). Every feature must be *deterministic* and *available at prediction time*—if a feature depends on information that only becomes available after delivery, using it would constitute data leakage.

```
# Use manage_dates for temporal feature extraction
df = manage_dates(df, columns=[&quot;order_datetime&quot;])

# Rename to shorter modeling column names
df = df.rename(columns={
  &quot;order_datetime_weekday&quot;: &quot;order_dow&quot;,
  &quot;order_datetime_hour&quot;: &quot;order_hour&quot;
})

# Customer age (derived from two dates — not a simple decomposition)
df[&quot;birthdate&quot;] = pd.to_datetime(df[&quot;birthdate&quot;])
df[&quot;customer_age&quot;] = (df[&quot;order_datetime&quot;] -
df[&quot;birthdate&quot;]).dt.days // 365

# Historical order volume per customer
df[&quot;customer_order_count&quot;] = (
  df.groupby(&quot;customer_id&quot;)[&quot;order_id&quot;]
    .transform(&quot;count&quot;)
)

df[[
  &quot;num_items&quot;,
  &quot;total_line_value&quot;,
  &quot;order_subtotal&quot;,
  &quot;shipping_fee&quot;,
  &quot;customer_age&quot;,
  &quot;customer_order_count&quot;
]].describe()


# Output:
#          num_items  total_line_value  order_subtotal  shipping_fee  customer_age
customer_order_count
# count      5000.0            5000.0          5000.0        5000.0        5000.0
5000.0
# mean          4.1             384.1           384.1           9.7          31.1
375.7
# std           2.2             284.1           284.1           5.1          14.4
449.2
```

```
      # min           1.0               4.7             4.7           0.0          18.0
1.0
      # max          12.0            1921.2          1921.2          28.1          75.0
1156.0
```

The *describe* output serves as a quick sanity check. Notice that *total_line_value* and *order_subtotal* have nearly identical distributions—expected because the subtotal is computed from line items. The wide range of *customer_order_count* (1 to 1,156) reflects a mix of new and highly active repeat customers. The *manage_dates* function also created *order_datetime_day*, *order_datetime_month*, and *order_datetime_year* columns that we do not include in the modeling table—not every generated feature needs to be used, and feature selection happens when you define the final column list.

Define the Modeling Target

For this chapter, the target variable is *late_delivery*, which the shipments table already provides. A value of 1 means actual delivery exceeded the promised delivery window.

```
      # The target comes directly from the shipments table
      # late_delivery = 1 means actual delivery exceeded promised days
      df[&quot;late_delivery&quot;].value_counts(normalize=True)


      # Output:
      # late_delivery
      # 1    0.596
      # 0    0.404
```

About 60% of orders experienced a late delivery, creating a moderate class imbalance. This matters for evaluation: a model that always predicts “late” would achieve roughly 60% accuracy without learning anything useful, so any model we build must significantly exceed that baseline to be worthwhile.

Shipment outcome columns (actual delivery days, carrier, shipping method) are intentionally excluded from the modeling features to avoid *data leakage*. Those values are not known at the time the order is placed, which is when predictions must be generated. If you included carrier or delivery_days as features, the model would appear to perform well during training but would fail in production because those columns would be empty for new, unshipped orders.

Load into the Analytical Database

Finally, we write the denormalized dataset into a separate SQLite database used exclusively for modeling.

```
# Select modeling columns
modeling_cols = [
  &quot;order_id&quot;, &quot;customer_id&quot;,
  &quot;num_items&quot;, &quot;num_distinct_products&quot;,
&quot;avg_unit_price&quot;,
  &quot;total_line_value&quot;, &quot;order_subtotal&quot;,
&quot;shipping_fee&quot;, &quot;promo_used&quot;,
  &quot;customer_age&quot;, &quot;customer_order_count&quot;,
  &quot;order_dow&quot;, &quot;order_hour&quot;,
  &quot;late_delivery&quot;
]

df_model = df[modeling_cols].dropna(subset=[&quot;late_delivery&quot;])

# Connect to analytical database
warehouse_conn = sqlite3.connect(&quot;warehouse.db&quot;)

# Write modeling table
df_model.to_sql(
  &quot;fact_orders_ml&quot;,
  warehouse_conn,
  if_exists=&quot;replace&quot;,
  index=False
)

warehouse_conn.close()
print(f&quot;warehouse.db created with table: fact_orders_ml ({len(df_model)}
rows)&quot;)


# Output:
# warehouse.db created with table: fact_orders_ml (5000 rows)
```

Two details are worth noting. First, *dropna(subset=["late_delivery"])* removes any orders without a shipment record (and therefore no label)—you cannot train on rows with unknown outcomes. Second, *to_sql* with *if_exists="replace"* means each ETL run produces a clean, complete table rather than appending incrementally. This keeps the pipeline simple and repeatable: run it again tomorrow and you get the same result (plus any new data).

Key Takeaways

This ETL step turns raw, normalized application data into a stable analytical asset. The operational database (shop.db) stores data optimized for the application; the analytical database (warehouse.db) stores data optimized for modeling.

Once created, *warehouse.db* becomes the *sole data source* for training and evaluation. This creates a clear contract: the training code never reaches back into shop.db, and the ETL code never touches the model. If a data issue arises, you know to look at the ETL step. If a model issue arises, you know to look at the training step.

This separation mirrors real deployment pipelines at any scale. The only difference between this chapter and a production data warehouse is the tooling—the architectural pattern is identical.

In the next section, you will build a training pipeline that assumes this data already exists and focuses entirely on modeling.

17.4Training

With the ETL pipeline complete, we can now focus entirely on model training. At this stage, the problem looks like any other supervised learning task you have seen earlier in this course.

This separation is intentional. The training code assumes that data cleaning and feature construction have already been handled upstream in the warehouse build step and does not reach back into the operational database.

Load the Modeling Data

We begin by loading the denormalized modeling table from the analytical database (the warehouse SQLite file). Each row represents one order, with engineered features and a labeled target.

```
import sqlite3
import pandas as pd

conn = sqlite3.connect(&quot;warehouse.db&quot;)

# Load the modeling table created by the ETL step
df = pd.read_sql(&quot;SELECT * FROM fact_orders_ml&quot;, conn)
conn.close()

print(df.shape)
df.head()


# Output:
# (5000, 14)
#    order_id  customer_id  num_items  ...  order_dow  order_hour  late_delivery
# 0         1            1          9  ...          5           0              1
# 1         2            1          7  ...          0          10              1
# 2         3            1          5  ...          0           7              1
```

The table has 5,000 rows and 14 columns: *order_id* and *customer_id* (identifiers that will not be used as features), 11 engineered features, and *late_delivery* (the target). This is the clean, stable dataset that the ETL step produced.

Select Features and Target

We now explicitly define which columns serve as features and which column is the target. Listing features by name (rather than selecting “everything except the target”) is a deployment best practice: it prevents accidental leakage if new columns are added to the warehouse table later, and it documents exactly what the model expects.

```
from sklearn.model_selection import train_test_split

label_col = &quot;late_delivery&quot;

feature_cols = [
  &quot;num_items&quot;,
  &quot;num_distinct_products&quot;,
  &quot;avg_unit_price&quot;,
  &quot;total_line_value&quot;,
  &quot;order_subtotal&quot;,
  &quot;shipping_fee&quot;,
  &quot;promo_used&quot;,
  &quot;customer_age&quot;,
  &quot;customer_order_count&quot;,
  &quot;order_dow&quot;,
  &quot;order_hour&quot;
]

X = df[feature_cols]
y = df[label_col].astype(int)

X_train, X_test, y_train, y_test = train_test_split(
  X, y,
  test_size=0.25,
  random_state=42,
  stratify=y
)

X_train.shape, X_test.shape


# Output:
# ((3750, 11), (1250, 11))
```

The 75/25 split gives us 3,750 rows for training and 1,250 for testing. The *stratify=y* argument ensures both sets preserve the same ratio of late-to-on-time deliveries as the original dataset. The *random_state=42* makes the split

reproducible: rerunning this cell always produces the same split. Notice what is *not* here: no ad-hoc cleaning, no feature creation, no data wrangling. All of that was handled in the ETL step so that training remains consistent, repeatable, and auditable.

Build a Training Pipeline

We now introduce scikit-learn's *Pipeline* object. A Pipeline packages preprocessing steps and the model into a single unit that can be saved to disk and reused during inference. This matters because the preprocessing steps (imputation values and scaling parameters) must be *learned from the training data* and then applied identically to new data. If you fit a scaler on training data but forget to apply it during inference, the model receives unscaled inputs and produces meaningless predictions.

The pipeline below has three steps: (1) a *SimpleImputer* that fills any missing values with the column median, (2) a *StandardScaler* that centers and scales each feature to zero mean and unit variance, and (3) a *LogisticRegression* classifier. When we save this pipeline to disk, all three components—including the learned medians and scaling parameters—are preserved together.

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline(steps=[
  (&quot;imputer&quot;, SimpleImputer(strategy=&quot;median&quot;)),
  (&quot;scaler&quot;, StandardScaler()),
  (&quot;model&quot;, LogisticRegression(max_iter=1000))
])

pipeline
```

```
# Output:
# Pipeline(steps=[('imputer', SimpleImputer(strategy='median')),
#                 ('scaler', StandardScaler()),
#                 ('model', LogisticRegression(max_iter=1000))])
```

Train the Model

Training the entire pipeline is a single method call. When you call *.fit()*, scikit-learn processes the steps in order: the imputer learns median values from the training data, the scaler learns means and standard deviations, and the logistic regression learns its coefficients. All of these learned parameters are stored inside the pipeline object.

```
pipeline.fit(X_train, y_train)
```

```
# Output:
# Pipeline(steps=[('imputer', SimpleImputer(strategy='median')),
#                 ('scaler', StandardScaler()),
#                 ('model', LogisticRegression(max_iter=1000))])
```

Evaluate Performance

Before saving the model, we must evaluate its performance on the held-out test set. This step is not optional—even in deployment contexts. Without evaluation, you have no evidence that the model is better than a coin flip (or better than always predicting the majority class).

```
from sklearn.metrics import classification_report, accuracy_score, f1_score,
roc_auc_score

y_pred = pipeline.predict(X_test)
y_prob = pipeline.predict_proba(X_test)[:, 1]
accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_prob)
```

```
report = classification_report(y_test, y_pred, output_dict=True)
accuracy, f1, roc_auc


# Output:
# (0.632, 0.7303634232121923, 0.6631244600970164)
```

The model achieves 63.2% accuracy, an F1 score of 0.73, and an ROC AUC of 0.66. These results are modest—this is a baseline logistic regression with no hyperparameter tuning. In a real project, you would experiment with stronger algorithms (Random Forest, Gradient Boosting) and more features. For this chapter, the model quality is secondary: the goal is to learn the deployment pipeline, not to maximize predictive performance.

In production systems, these metrics should always be recorded alongside the model so performance can be compared across retraining runs. A model that degrades over time is a clear signal of data drift or concept drift, which the next chapter covers in detail.

Save the Model Artifact

A trained model is just a file. We serialize the entire pipeline using *joblib*, which preserves preprocessing and modeling logic together.

```
import joblib

joblib.dump(pipeline, &quot;late_delivery_model.sav&quot;)


# Output:
# ['late_delivery_model.sav']
```

The *joblib.dump* call writes the entire pipeline—imputer, scaler, and logistic regression—into a single *.sav* file. This file can now be loaded by any Python process that has scikit-learn installed. The preprocessing parameters (median

values, scaling factors) are baked into the file alongside the model weights, so the loaded pipeline will transform new data exactly the same way it transformed the training data. That is why we say: *Your model is a file*.

Save Metrics and Metadata

Production models should always be accompanied by metadata (the training context) and metrics (the evaluation results). Without these files, you cannot answer basic questions later: *When was this model trained? How many rows did it see? Which features did it use? How well did it perform?* Saving this information alongside the model makes it auditable, traceable, and reproducible.

```
import json
from datetime import datetime, timezone

model_version = &quot;1.0.0&quot;

metadata = {
  &quot;model_name&quot;: &quot;late_delivery_pipeline&quot;,
  &quot;model_version&quot;: model_version,
  &quot;trained_at_utc&quot;: datetime.now(timezone.utc).isoformat(),
  &quot;warehouse_table&quot;: &quot;fact_orders_ml&quot;,
  &quot;num_training_rows&quot;: int(X_train.shape[0]),
  &quot;num_test_rows&quot;: int(X_test.shape[0]),
  &quot;features&quot;: feature_cols
}

metrics = {
  &quot;accuracy&quot;: float(accuracy),
  &quot;f1&quot;: float(f1),
  &quot;roc_auc&quot;: float(roc_auc),
  &quot;classification_report&quot;: report
}

with open(&quot;model_metadata.json&quot;, &quot;w&quot;, encoding=&quot;utf-8&quot;) as f:
  json.dump(metadata, f, indent=2)

with open(&quot;metrics.json&quot;, &quot;w&quot;, encoding=&quot;utf-8&quot;) as f:
  json.dump(metrics, f, indent=2)
```

What Gets Deployed

In this architecture, deployment means the application can reliably access the following artifacts:

- *late_delivery_model.sav* — the trained pipeline (preprocessing + model)
- *model_metadata.json* — version, timestamp, row counts, and feature list
- *metrics.json* — evaluation metrics and classification report

The scheduler may retrain this model on a schedule, but the application does not retrain. The application simply loads the latest model file or consumes predictions written back to the database.

Key Idea

Once the ETL step produces a consistent modeling table, training becomes fully repeatable and automatable.

Your model is a file.

In the next section, you will load this saved pipeline, generate late-delivery probabilities for live orders, and write predictions back into the operational database to support a warehouse priority workflow.

17.5Inference

Once a model has been trained and saved, deployment shifts from learning to prediction. This phase is called *inference*. In the training section, you built a model from historical data. In this section, you will use that saved model to generate predictions on *new, unseen* orders from the live operational database.

Inference does not involve training, gradients, or optimization. The model is loaded from disk as a read-only object and used to produce predictions. The key challenge is ensuring that the feature engineering applied to new data is *identical* to what was applied during training. Any mismatch—a missing column, a different aggregation, or a new category—will produce unreliable predictions.

Load the Trained Model

We begin by loading the serialized model artifact produced during training. Because the pipeline contains both preprocessing steps and the classifier, this single *joblib.load* call gives us a fully operational prediction engine.

```
import joblib

model = joblib.load(&quot;late_delivery_model.sav&quot;)
model


# Output:
# Pipeline(steps=[('imputer', SimpleImputer(strategy='median')),
#                 ('scaler', StandardScaler()),
#                 ('model', LogisticRegression(max_iter=1000))])
```

The printed representation confirms that the loaded object is the same three-step pipeline we trained earlier: imputer, scaler, and logistic regression. The preprocessing parameters (learned medians and scaling factors) are embedded inside the object.

Load New Orders from the Operational Database

A critical distinction: during inference, predictions are generated from *live operational data* (shop.db), not from the analytical warehouse. The warehouse was built for training; the operational database is where new

orders arrive. In production, you would typically score only orders that have not yet been shipped—identified by having no matching record in the shipments table. Since our sample database has shipped all orders, we score the 200 most recent orders to demonstrate the mechanism.

```
import sqlite3
import pandas as pd

conn = sqlite3.connect(&quot;shop.db&quot;)

# In production, score only orders without a shipment record:
#   LEFT JOIN shipments s ON s.order_id = o.order_id
#   WHERE s.shipment_id IS NULL
# In the provided database all orders have been shipped,
# so we score the 200 most recent orders for demonstration.
query = &quot;&quot;&quot;
SELECT
  o.order_id,
  o.customer_id,
  o.order_datetime,
  o.order_subtotal,
  o.shipping_fee,
  o.promo_used,
  c.birthdate

FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
ORDER BY o.order_datetime DESC
LIMIT 200
&quot;&quot;&quot;
df_live = pd.read_sql(query, conn)

print(f&quot;Orders to score: {len(df_live)}&quot;)
df_live.head()


# Output:
# Orders to score: 200
#    order_id  customer_id       order_datetime  order_subtotal  shipping_fee
promo_used   birthdate
# 0      4998          207  2026-01-14 23:48:30          296.45         12.99
0  1978-09-13
# 1      4994           74  2026-01-14 22:32:29          130.05          6.99
0  1981-02-01
```

Notice that this query pulls raw columns (order_subtotal, shipping_fee, promo_used, birthdate) rather than pre-computed features. That is intentional: the feature engineering logic must be applied explicitly in the

next step to ensure consistency with training. The SQL query is deliberately simple—it just extracts the raw ingredients.

Feature Engineering at Inference Time

This is the most error-prone step in any deployment pipeline. The features computed here must be *exactly the same* as those computed during ETL: same aggregation logic, same column names, same derived calculations. If even one feature is computed differently, the model will produce unreliable predictions without raising an error. In production systems, this logic should be extracted into shared functions imported by both the ETL and inference scripts.

```
from datetime import datetime, timezone

# 1. Aggregate order items (same logic as ETL)
oi = pd.read_sql(&quot;SELECT * FROM order_items&quot;, conn)

oi_agg = (
  oi[oi[&quot;order_id&quot;].isin(df_live[&quot;order_id&quot;])]
    .groupby(&quot;order_id&quot;)
    .agg(
      num_items=(&quot;quantity&quot;, &quot;sum&quot;),
      num_distinct_products=(&quot;product_id&quot;, &quot;nunique&quot;),
      avg_unit_price=(&quot;unit_price&quot;, &quot;mean&quot;),
      total_line_value=(&quot;line_total&quot;, &quot;sum&quot;)
    )
    .reset_index()
)

df_live = df_live.merge(oi_agg, on=&quot;order_id&quot;, how=&quot;left&quot;)

# 2. Derived features (consistent with ETL logic)
df_live[&quot;order_datetime&quot;] =
pd.to_datetime(df_live[&quot;order_datetime&quot;])
df_live[&quot;birthdate&quot;] = pd.to_datetime(df_live[&quot;birthdate&quot;])

df_live[&quot;customer_age&quot;] = (
  (df_live[&quot;order_datetime&quot;] - df_live[&quot;birthdate&quot;]).dt.days //
365
)

df_live[&quot;order_dow&quot;] = df_live[&quot;order_datetime&quot;].dt.dayofweek
df_live[&quot;order_hour&quot;] = df_live[&quot;order_datetime&quot;].dt.hour

# 3. Historical order count per customer
```

```
order_counts = pd.read_sql(
  &quot;SELECT customer_id, COUNT(*) AS customer_order_count FROM orders GROUP BY
customer_id&quot;,
  conn
)

df_live = df_live.merge(order_counts, on=&quot;customer_id&quot;,
how=&quot;left&quot;)

# 4. Build feature matrix
feature_cols = [
  &quot;num_items&quot;, &quot;num_distinct_products&quot;,
&quot;avg_unit_price&quot;,
  &quot;total_line_value&quot;, &quot;order_subtotal&quot;,
&quot;shipping_fee&quot;, &quot;promo_used&quot;,
  &quot;customer_age&quot;, &quot;customer_order_count&quot;, &quot;order_dow&quot;,
&quot;order_hour&quot;
]

X_live = df_live[feature_cols]
```

Compare this code to the ETL feature engineering: the aggregation (step 1), the date-derived features (step 2), and the historical count (step 3) all mirror the ETL logic exactly. The feature list in step 4 matches the 11 features used during training. If you added or removed a feature in ETL, you would need to update this code as well—which is exactly why production teams extract shared feature logic into reusable modules.

Generate Predictions

With the feature matrix built, we can now call the model. The *predict_proba* method returns a probability for each class; we take column index 1 to get the probability of late delivery. The *predict* method returns the binary classification (0 or 1) using the default 0.5 threshold.

```
df_live[&quot;late_delivery_prob&quot;] = model.predict_proba(X_live)[:, 1]
df_live[&quot;late_delivery_pred&quot;] = model.predict(X_live)
df_live[[&quot;order_id&quot;, &quot;late_delivery_prob&quot;,
&quot;late_delivery_pred&quot;]].head()


# Output:
#    order_id  late_delivery_prob  late_delivery_pred
# 0      4998            0.582417                   1
```

```
# 1      4994            0.619234                  1
# 2      4989            0.573891                  1
# 3      4988            0.551029                  1
# 4      4985            0.642580                  1
```

Each order now has both a probability (useful for ranking) and a binary prediction (useful for filtering). In practice, probabilities are far more valuable than binary labels for operational decision-making. A warehouse manager can set their own threshold: “prioritize any order above 70% risk” or “flag the top 20 riskiest orders.”

Write Predictions Back to the Operational Database

Predictions are most valuable when written back into systems that drive real action. The code below creates a dedicated *order_predictions* table in the operational database and inserts one row per scored order. The *INSERT OR REPLACE* pattern ensures that re-running inference updates existing predictions rather than creating duplicates.

```
cursor = conn.cursor()

cursor.execute(&quot;&quot;&quot;
CREATE TABLE IF NOT EXISTS order_predictions (
  order_id INTEGER PRIMARY KEY,
  late_delivery_probability REAL,
  predicted_late_delivery INTEGER,
  prediction_timestamp TEXT
)
&quot;&quot;&quot;)

rows = [
  (
    int(row.order_id),
    float(row.late_delivery_prob),
    int(row.late_delivery_pred),
    datetime.now(timezone.utc).isoformat()
  )
  for row in df_live.itertuples()
]

cursor.executemany(&quot;&quot;&quot;
INSERT OR REPLACE INTO order_predictions
(order_id, late_delivery_probability, predicted_late_delivery, prediction_timestamp)
VALUES (?, ?, ?, ?)
```

```
&quot;&quot;&quot;, rows)
conn.commit()
conn.close()
```

The operational database now contains model output alongside transactional data. The *prediction_timestamp* column records when each prediction was generated, which is essential for auditing and for detecting stale predictions that should be refreshed.

Using Predictions in the Application

This is the payoff of the write-back pattern. The web application does not need to import scikit-learn, load a model, or understand machine learning at all. It simply queries a table that already contains predictions, just like any other application query:

```
SELECT *
FROM order_predictions
ORDER BY late_delivery_probability DESC;
```

This enables a warehouse dashboard to prioritize orders most likely to be delayed.

Key Deployment Pattern

- Models are trained offline
- Predictions are written into operational systems
- Applications consume predictions like any other data

This pattern avoids embedding ML logic directly into application code.

Key Idea

Inference is not about intelligence. It is about integration.

A deployed model is useful only when its predictions influence real decisions.

In the final section, we will reflect on how this pipeline mirrors real-world deployment—and when simpler solutions are preferable.

17.6Scheduled Jobs

So far, you have developed the pipeline in notebooks so you can experiment, inspect intermediate outputs, and debug quickly. In real deployments, however, these same steps must run automatically on a schedule.

In this chapter, we will convert the core logic into a small set of Python scripts (.py files). These scripts will be executed in a repeating cycle:

- **ETL** creates a denormalized modeling table in a separate SQLite warehouse file.
- **Training** trains a model from the warehouse table, saves the model file, and saves metadata and metrics.
- **Inference** loads the saved model and writes predictions back into the operational database for the application to use.

This structure helps you see the difference between analytics work (in notebooks) and production work (scheduled jobs). Each job is reusable code that can be run manually for debugging or run automatically for deployment.

Project Folder Layout

Create a folder for your project with the following structure:

```
project/
  data/
    shop.db
    warehouse.db

  artifacts/
    late_delivery_model.sav
    model_metadata.json
    metrics.json

  jobs/
    config.py
    utils_db.py
    etl_build_warehouse.py
    train_model.py
    run_inference.py
```

The **data** folder holds your operational database (shop.db) and your simplified warehouse database (warehouse.db). The **artifacts** folder holds outputs produced by training.

Shared Configuration

All jobs should agree on paths and filenames. Put shared paths in one place, and make sure the artifacts folder exists before saving outputs.

```
# jobs/config.py
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = PROJECT_ROOT / &quot;data&quot;
ARTIFACTS_DIR = PROJECT_ROOT / &quot;artifacts&quot;
OP_DB_PATH = DATA_DIR / &quot;shop.db&quot;
WH_DB_PATH = DATA_DIR / &quot;warehouse.db&quot;
MODEL_PATH = ARTIFACTS_DIR / &quot;late_delivery_model.sav&quot;
MODEL_METADATA_PATH = ARTIFACTS_DIR / &quot;model_metadata.json&quot;
METRICS_PATH = ARTIFACTS_DIR / &quot;metrics.json&quot;
```

Database Utilities

These helpers keep database code consistent across ETL, training, and inference.

```
# jobs/utils_db.py
import sqlite3
from contextlib import contextmanager

@contextmanager

def sqlite_conn(db_path):
  conn = sqlite3.connect(str(db_path))
  try:
    yield conn
  finally:
    conn.close()

def ensure_predictions_table(conn):
  cur = conn.cursor()

  cur.execute(&quot;&quot;&quot;
  CREATE TABLE IF NOT EXISTS order_predictions (
    order_id INTEGER PRIMARY KEY,
    late_delivery_probability REAL,
    predicted_late_delivery INTEGER,
    prediction_timestamp TEXT
  )
  &quot;&quot;&quot;)
  conn.commit()
```

Job 1: ETL to Build the Warehouse

This job reads operational tables and writes a denormalized modeling table into a separate SQLite database (warehouse.db). If your table or column names differ, adjust the SQL query so it matches your shop.db schema.

```
# jobs/etl_build_warehouse.py
import pandas as pd
from config import OP_DB_PATH, WH_DB_PATH
from utils_db import sqlite_conn

def build_modeling_table():
  with sqlite_conn(OP_DB_PATH) as conn:
    orders = pd.read_sql(&quot;SELECT * FROM orders&quot;, conn)
    customers = pd.read_sql(&quot;SELECT * FROM customers&quot;, conn)
    order_items = pd.read_sql(&quot;SELECT * FROM order_items&quot;, conn)
```

```
          shipments = pd.read_sql(&quot;SELECT order_id, late_delivery FROM
shipments&quot;, conn)
        # Aggregate order items
        oi_agg = (
          order_items.groupby(&quot;order_id&quot;)
            .agg(
              num_items=(&quot;quantity&quot;, &quot;sum&quot;),
              num_distinct_products=(&quot;product_id&quot;, &quot;nunique&quot;),
              avg_unit_price=(&quot;unit_price&quot;, &quot;mean&quot;),
              total_line_value=(&quot;line_total&quot;, &quot;sum&quot;)
            )
            .reset_index()
        )

        # Denormalize
        df = (
          orders
            .merge(customers, on=&quot;customer_id&quot;, how=&quot;left&quot;)
            .merge(oi_agg, on=&quot;order_id&quot;, how=&quot;left&quot;)
            .merge(shipments, on=&quot;order_id&quot;, how=&quot;left&quot;)
        )

        # Feature engineering
        df[&quot;order_datetime&quot;] = pd.to_datetime(df[&quot;order_datetime&quot;],
errors=&quot;coerce&quot;)
        df[&quot;birthdate&quot;] = pd.to_datetime(df[&quot;birthdate&quot;],
errors=&quot;coerce&quot;)
        df[&quot;customer_age&quot;] = (df[&quot;order_datetime&quot;] -
df[&quot;birthdate&quot;]).dt.days // 365
        df[&quot;order_dow&quot;] = df[&quot;order_datetime&quot;].dt.dayofweek
        df[&quot;order_hour&quot;] = df[&quot;order_datetime&quot;].dt.hour

        df[&quot;customer_order_count&quot;] = (
          df.groupby(&quot;customer_id&quot;)
[&quot;order_id&quot;].transform(&quot;count&quot;)
        )

        modeling_cols = [
          &quot;order_id&quot;, &quot;customer_id&quot;,
          &quot;num_items&quot;, &quot;num_distinct_products&quot;,
&quot;avg_unit_price&quot;,
          &quot;total_line_value&quot;, &quot;order_subtotal&quot;,
&quot;shipping_fee&quot;, &quot;promo_used&quot;,
          &quot;customer_age&quot;, &quot;customer_order_count&quot;,
          &quot;order_dow&quot;, &quot;order_hour&quot;,
          &quot;late_delivery&quot;
        ]

        df_model = df[modeling_cols].dropna(subset=[&quot;late_delivery&quot;])
        with sqlite_conn(WH_DB_PATH) as wh_conn:
          df_model.to_sql(&quot;modeling_orders&quot;, wh_conn,
if_exists=&quot;replace&quot;, index=False)

        return len(df_model)

      if __name__ == &quot;__main__&quot;:
        row_count = build_modeling_table()
        print(f&quot;Warehouse updated. modeling_orders rows: {row_count}&quot;)
```

Job 2: Train the Model and Save Artifacts

This job trains the model from the warehouse table and writes three outputs to disk:

- **late_delivery_model.sav** (the trained model file)
- **model_metadata.json** (version, timestamp, row counts, feature list)
- **metrics.json** (evaluation metrics)

```
# jobs/train_model.py
import json
from datetime import datetime, timezone
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score
from sklearn.linear_model import LogisticRegression
from config import WH_DB_PATH, ARTIFACTS_DIR, MODEL_PATH, MODEL_METADATA_PATH,
METRICS_PATH
from utils_db import sqlite_conn

MODEL_VERSION = &quot;1.0.0&quot;

FEATURE_COLS = [
  &quot;num_items&quot;, &quot;num_distinct_products&quot;,
&quot;avg_unit_price&quot;,
  &quot;total_line_value&quot;, &quot;order_subtotal&quot;,
&quot;shipping_fee&quot;, &quot;promo_used&quot;,
  &quot;customer_age&quot;, &quot;customer_order_count&quot;, &quot;order_dow&quot;,
&quot;order_hour&quot;
]

def train_and_save():
  with sqlite_conn(WH_DB_PATH) as conn:
    df = pd.read_sql(&quot;SELECT * FROM modeling_orders&quot;, conn)

  label_col = &quot;late_delivery&quot;
  X = df[FEATURE_COLS]
  y = df[label_col].astype(int)

  X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
  )

  model = Pipeline(steps=[
    (&quot;imputer&quot;, SimpleImputer(strategy=&quot;median&quot;)),
    (&quot;scaler&quot;, StandardScaler()),
    (&quot;clf&quot;, LogisticRegression(max_iter=1000))
  ])

  model.fit(X_train, y_train)
```

```
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    metrics = {
      &quot;accuracy&quot;: float(accuracy_score(y_test, y_pred)),
      &quot;f1&quot;: float(f1_score(y_test, y_pred)),
      &quot;roc_auc&quot;: float(roc_auc_score(y_test, y_prob)),
      &quot;row_count_train&quot;: int(len(X_train)),
      &quot;row_count_test&quot;: int(len(X_test))
    }

    ARTIFACTS_DIR.mkdir(parents=True, exist_ok=True)
    joblib.dump(model, str(MODEL_PATH))

    metadata = {
      &quot;model_version&quot;: MODEL_VERSION,
      &quot;trained_at_utc&quot;: datetime.now(timezone.utc).isoformat(),
      &quot;feature_list&quot;: FEATURE_COLS,
      &quot;label&quot;: label_col,
      &quot;warehouse_table&quot;: &quot;modeling_orders&quot;,
      &quot;warehouse_rows&quot;: int(len(df))
    }

    with open(MODEL_METADATA_PATH, &quot;w&quot;, encoding=&quot;utf-8&quot;) as f:
      json.dump(metadata, f, indent=2)

    with open(METRICS_PATH, &quot;w&quot;, encoding=&quot;utf-8&quot;) as f:
      json.dump(metrics, f, indent=2)

    print(&quot;Training complete.&quot;)
    print(f&quot;Saved model: {MODEL_PATH}&quot;)
    print(f&quot;Saved metadata: {MODEL_METADATA_PATH}&quot;)
    print(f&quot;Saved metrics: {METRICS_PATH}&quot;)

  if __name__ == &quot;__main__&quot;:
    train_and_save()
```

Job 3: Run Inference and Write Predictions to shop.db

This job loads the latest saved model and generates predictions for orders that do not yet have a shipment record. It then writes predictions to a dedicated table keyed by order_id.

```
  # jobs/run_inference.py
  import pandas as pd
  import joblib
  from datetime import datetime, timezone
  from config import OP_DB_PATH, MODEL_PATH
  from utils_db import sqlite_conn, ensure_predictions_table

  FEATURE_COLS = [
```

```
        &quot;num_items&quot;, &quot;num_distinct_products&quot;,
&quot;avg_unit_price&quot;,
        &quot;total_line_value&quot;, &quot;order_subtotal&quot;,
&quot;shipping_fee&quot;, &quot;promo_used&quot;,
        &quot;customer_age&quot;, &quot;customer_order_count&quot;, &quot;order_dow&quot;,
&quot;order_hour&quot;
      ]

      def run_inference():
        model = joblib.load(str(MODEL_PATH))
        with sqlite_conn(OP_DB_PATH) as conn:
          # Score orders that have no shipment record yet
          orders = pd.read_sql(&quot;&quot;&quot;
            SELECT o.order_id, o.customer_id, o.order_datetime,
                   o.order_subtotal, o.shipping_fee, o.promo_used,
                   c.birthdate
            FROM orders o
            JOIN customers c ON o.customer_id = c.customer_id
            LEFT JOIN shipments s ON s.order_id = o.order_id
            WHERE s.shipment_id IS NULL
          &quot;&quot;&quot;, conn)

          if len(orders) == 0:
            print(&quot;No unshipped orders to score.&quot;)

            return
          # Aggregate order items
          oi = pd.read_sql(&quot;SELECT * FROM order_items&quot;, conn)

          oi_agg = (
            oi[oi[&quot;order_id&quot;].isin(orders[&quot;order_id&quot;])]
              .groupby(&quot;order_id&quot;)
              .agg(
                num_items=(&quot;quantity&quot;, &quot;sum&quot;),
                num_distinct_products=(&quot;product_id&quot;, &quot;nunique&quot;),
                avg_unit_price=(&quot;unit_price&quot;, &quot;mean&quot;),
                total_line_value=(&quot;line_total&quot;, &quot;sum&quot;)
              )
              .reset_index()
          )

          # Customer order counts
          counts = pd.read_sql(
            &quot;SELECT customer_id, COUNT(*) AS customer_order_count FROM orders GROUP
BY customer_id&quot;,
            conn
          )

        df = orders.merge(oi_agg, on=&quot;order_id&quot;,
how=&quot;left&quot;).merge(counts, on=&quot;customer_id&quot;, how=&quot;left&quot;)
        df[&quot;order_datetime&quot;] = pd.to_datetime(df[&quot;order_datetime&quot;],
errors=&quot;coerce&quot;)
        df[&quot;birthdate&quot;] = pd.to_datetime(df[&quot;birthdate&quot;],
errors=&quot;coerce&quot;)
        df[&quot;customer_age&quot;] = (df[&quot;order_datetime&quot;] -
df[&quot;birthdate&quot;]).dt.days // 365
        df[&quot;order_dow&quot;] = df[&quot;order_datetime&quot;].dt.dayofweek
        df[&quot;order_hour&quot;] = df[&quot;order_datetime&quot;].dt.hour
        X_live = df[FEATURE_COLS]
        probs = model.predict_proba(X_live)[:, 1]
        preds = model.predict(X_live)
        ts = datetime.now(timezone.utc).isoformat()
```

```
        out_rows = [
          (int(oid), float(p), int(yhat), ts)

          for oid, p, yhat in zip(df[&quot;order_id&quot;], probs, preds)
        ]

        with sqlite_conn(OP_DB_PATH) as conn:
          ensure_predictions_table(conn)
          cur = conn.cursor()

          cur.executemany(&quot;&quot;&quot;
          INSERT OR REPLACE INTO order_predictions
          (order_id, late_delivery_probability, predicted_late_delivery,
prediction_timestamp)
          VALUES (?, ?, ?, ?)
          &quot;&quot;&quot;, out_rows)
          conn.commit()

        print(f&quot;Inference complete. Predictions written: {len(out_rows)}&quot;)

      if __name__ == &quot;__main__&quot;:
        run_inference()
```

How These Scripts Run Automatically

In production, these scripts are executed by a scheduler. The scheduler is not "AI." It is simply a timed process that runs commands repeatedly.

A realistic schedule for this project could be:

- ETL runs every night (build modeling table)
- Training runs every night after ETL
- Inference runs every few minutes (keep predictions fresh)

Cron Scheduling Example (Mac or Linux)

Cron is a common scheduler on Unix-like systems. You edit your scheduled commands using:

```
      crontab -e
```

Cron runs in a minimal environment. Use absolute paths and, if you are using a virtual environment, activate it explicitly in the command.

```
      # Nightly ETL at 1:00am
      0 1 * * * cd /path/to/project &amp;&amp; /path/to/venv/bin/python
jobs/etl_build_warehouse.py &gt;&gt; logs/etl.log 2&gt;&amp;1

      # Nightly training at 1:10am
      10 1 * * * cd /path/to/project &amp;&amp; /path/to/venv/bin/python
jobs/train_model.py &gt;&gt; logs/train.log 2&gt;&amp;1

      # Inference every 5 minutes
      */5 * * * * cd /path/to/project &amp;&amp; /path/to/venv/bin/python
jobs/run_inference.py &gt;&gt; logs/infer.log 2&gt;&amp;1
```

In cron syntax, the five fields represent minute, hour, day-of-month, month, and day-of-week.

Windows Scheduling Option

On Windows, you can use Task Scheduler to run the same commands on a repeating trigger. The important concept is unchanged: a scheduler runs your scripts automatically at the times you define.

Alternative Scheduling for the Vibe-Coded App

Later, in the app-building section, you will see an additional scheduling option that can be easier for student projects: running scheduled jobs inside the application process using a lightweight job runner. This is useful for demos and learning, but OS-level scheduling is still the most reliable pattern.

- **Node/JavaScript**: node-cron (simple) or a background worker (more robust).
- **Python**: APScheduler (runs scheduled Python functions or commands).

- **ASP.NET/C#**: Quartz.NET or Hangfire (background jobs with dashboards).

Your web application does not retrain the model. Instead, it reads predictions written into shop.db and uses them to prioritize orders.

A simple “warehouse priority list” query can sort by late_delivery_probability descending and show the top orders to fulfill first.

17.7Vibe Code Priority Queue

What the App Needs

At this point, training produces a model file and metrics. In a production application, the next step is to turn predictions into an operational workflow feature. Here, that feature is a warehouse *priority queue*: orders with the highest predicted probability of late delivery rise to the top so the warehouse can process them first.

To keep the app simple, we treat the operational database as the system of record and write predictions back to it. The app then reads a single query (or a database view, if you choose to create one) to display the priority queue. The key deployment idea is that the app does not run machine learning code—it just reads predictions like any other table.

Single Query the App Uses

The app can use the following SQL query to return a prioritized list of orders. This query assumes you have a table named *order_predictions* keyed by

order_id with columns *late_delivery_probability*, *predicted_late_delivery*, and *prediction_timestamp*.

```
SELECT
  o.order_id,
  o.order_datetime,
  o.order_total,
  c.customer_id,
  c.full_name AS customer_name,
  p.late_delivery_probability,
  p.predicted_late_delivery,
  p.prediction_timestamp

FROM orders o
JOIN customers c
  ON c.customer_id = o.customer_id

JOIN order_predictions p
  ON p.order_id = o.order_id

LEFT JOIN shipments s
  ON s.order_id = o.order_id

WHERE s.shipment_id IS NULL
ORDER BY
  p.late_delivery_probability DESC,
  o.order_datetime ASC

LIMIT 50;
```

Operationally, this becomes the “next orders to pull” list. The only thing the UI must do is run this query and render the result in a table.

Using Cursor to Build the UI Feature

You will use Cursor (Education) to generate most of the application scaffolding. Your job is to provide the AI agent with clear requirements and a stable database contract. Use one of the three stacks below, based on your prior coding experience.

- If you have little or no web development background, use the *JavaScript stack* (Next.js) for the most guided path.

- If you are strongest in Python, use the *Python stack* (FastAPI) to keep everything in one language.
- If you have C# experience, use the *ASP.NET stack* (minimal APIs or MVC) for a professional enterprise-style pattern.

AI Prompts Students Can Paste

Pick one prompt below and paste it into Cursor (or Claude Code). After the agent generates code, you will run the app, validate the priority page loads, and confirm it returns the same rows as the SQL query above.

Option A: JavaScript (Next.js) Prompt

```
      You are building a small student project web app using Next.js (App Router) and a
SQLite database named &quot;shop.db&quot;.
      Requirements:
      1. Create a page at /warehouse/priority that displays a &quot;Late Delivery Priority
Queue&quot; table.
      2. The page must run this SQL query against shop.db and render the results:
      SELECT
        o.order_id,
        o.order_datetime,
        o.order_total,
        c.customer_id,
        c.full_name AS customer_name,
        p.late_delivery_probability,
        p.predicted_late_delivery,
        p.prediction_timestamp

      FROM orders o
      JOIN customers c ON c.customer_id = o.customer_id
      JOIN order_predictions p ON p.order_id = o.order_id
      LEFT JOIN shipments s ON s.order_id = o.order_id
      WHERE s.shipment_id IS NULL
      ORDER BY p.late_delivery_probability DESC, o.order_datetime ASC
      LIMIT 50;
      3. Use a lightweight SQLite library for Node (better-sqlite3 preferred).
      4. Create a simple layout with a header, a short explanatory paragraph, and a table
(sortable columns optional).
      5. Include minimal styling (clean, readable). No authentication required.
      Deliverables:
      - All code changes needed in the Next.js project
      - Any install commands (npm) and how to run
      - A short note explaining where shop.db should be located in the repo
```

Option B: Python (FastAPI) Prompt

You are building a small student project web app using Python FastAPI and Jinja2 templates with a SQLite database named "shop.db".

Requirements:

1. Create a route GET /warehouse/priority that renders an HTML page titled "Late Delivery Priority Queue".
2. The route must run this SQL query against shop.db and render the results in an HTML table:

```
SELECT
  o.order_id,
  o.order_datetime,
  o.order_total,
  c.customer_id,
  c.full_name AS customer_name,
  p.late_delivery_probability,
  p.predicted_late_delivery,
  p.prediction_timestamp

FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN order_predictions p ON p.order_id = o.order_id
LEFT JOIN shipments s ON s.order_id = o.order_id
WHERE s.shipment_id IS NULL
ORDER BY p.late_delivery_probability DESC, o.order_datetime ASC
LIMIT 50;
```

3. Use the built-in sqlite3 module (no ORM).
4. Provide minimal styling and a clean table layout.
5. Include instructions to run with uvicorn.

Deliverables:

- Project structure (main.py, templates, static if needed)
- pip install commands
- How to run and where to place shop.db

Option C: ASP.NET/C# Prompt

You are building a small student project web app using ASP.NET Core (minimal API or MVC) and SQLite database "shop.db".

Requirements:

1. Create an endpoint /warehouse/priority that returns an HTML page titled "Late Delivery Priority Queue".
2. The endpoint must run this SQL query against shop.db and render results in a table:

```
SELECT
  o.order_id,
  o.order_datetime,
  o.order_total,
  c.customer_id,
  c.full_name AS customer_name,
  p.late_delivery_probability,
  p.predicted_late_delivery,
  p.prediction_timestamp

FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN order_predictions p ON p.order_id = o.order_id
LEFT JOIN shipments s ON s.order_id = o.order_id
WHERE s.shipment_id IS NULL
ORDER BY p.late_delivery_probability DESC, o.order_datetime ASC
LIMIT 50;
```

```
3. Use Microsoft.Data.Sqlite to query (no Entity Framework required).
4. Provide minimal CSS styling and clear layout.
5. Provide commands to run locally (dotnet run) and where shop.db should live.
Deliverables:
- Full code changes (Program.cs plus any views/templates if used)
- NuGet package install commands
- Run instructions
```

What to Check

- The page loads without errors and the table renders.
- The top rows have the highest *late_delivery_probability*.
- The result matches running the same SQL directly in a SQLite browser.

In the next section, you will run inference on new orders and write predictions into *order_predictions* so the app can display this priority queue.

17.8Vibe Code App

Goal

In this chapter, you built a realistic ML pipeline that reads from a live operational database, creates an analytical “warehouse” table for modeling, trains a model, saves a model artifact, generates predictions, and writes those predictions back to the operational database.

In this section, you will *vibe code* a complete (but simple) web app on top of that database. You will use an AI coding agent (Cursor or Claude Code) to generate most of the application scaffolding. Your job is to (1) provide clear requirements, (2) keep the database contract stable, and (3) test the application until it matches expected behavior.

To keep scope manageable, this app intentionally ignores authentication. Instead, it lets the user select an existing customer to “act as” during testing.

What Your App Must Do

- Use an existing SQLite database file named *shop.db* (operational DB).
- Provide a “Select Customer” screen (no signup/login).
- Allow placing a new order for the selected customer.
- Save the order + line items into *shop.db*.
- Show an order history page for that customer.
- Show the warehouse “Late Delivery Priority Queue” page (top 50).
- Provide a “Run Scoring” button that triggers the inference job and then refreshes the priority queue.

You will build the app in one of three stacks. The recommended default is JavaScript (Next.js) because it is widely supported by AI coding agents and has a straightforward developer experience.

Database Contract

Your AI agent must not invent new tables. It should only use the operational database tables you already have (for example, *customers*, *orders*, *order_items*, *products*, and *order_predictions*). If your database uses different table or column names, update the prompts below to match your schema.

The pipeline writes predictions into *order_predictions* keyed by *order_id*. The application should treat that table like any other application table.

Recommended Stack

For students with limited background, use: *Next.js + SQLite* for the web app, and a separate *Python inference script* that writes predictions into the database.

The rest of this section provides a complete sequence of copy/paste prompts. Paste them into Cursor (or Claude Code) in order. After each step, run the app and verify behavior before moving on.

Prompt 0: Project Setup (Next.js)

```
You are generating a complete student project web app using Next.js (App Router) and
SQLite.
Constraints:
- No authentication. Users select an existing customer to &quot;act as&quot;.
- Use a SQLite file named &quot;shop.db&quot; located at the project root (or
/data/shop.db if you prefer).
- Use better-sqlite3 for DB access.
- Keep UI simple and clean.
Tasks:
1. Create a new Next.js app (App Router).
2. Add a server-side DB helper module that opens shop.db and exposes helpers for
SELECT and INSERT/UPDATE using prepared statements.
3. Create a shared layout with navigation links:
  - Select Customer
  - Customer Dashboard
  - Place Order
  - Order History
  - Warehouse Priority Queue
  - Run Scoring

4. Provide install/run instructions (npm) and any required scripts.
Return:
- All files to create/modify
- Any commands to run
```

Prompt 0.5: Inspect the Database Schema

```
Add a developer-only page at /debug/schema that prints:
- All table names in shop.db
- For each table, the column names and types (PRAGMA table_info)
Purpose: Students can verify the real schema and adjust prompts if needed.
Keep it simple and readable.
```

Prompt 1: Select Customer Screen

```
Add a &quot;Select Customer&quot; page at /select-customer.
Requirements:
1. Query the database for customers:
  - customer_id
  - full_name
  - email

2. Render a searchable dropdown or simple list. When a customer is selected, store
customer_id in a cookie.
3. Redirect to /dashboard after selection.
4. Add a small banner showing the currently selected customer on every page (if
set).
Deliver:
- Any new routes/components
- DB query code using better-sqlite3
- Notes on where customer_id is stored
```

Prompt 2: Customer Dashboard

```
Create a /dashboard page that shows a summary for the selected customer.
Requirements:
1. If no customer is selected, redirect to /select-customer.
2. Show:
  - Customer name and email
  - Total number of orders for the customer
  - Total spend across all orders (sum order_total)
  - A small table of the 5 most recent orders (order_id, order_datetime,
order_total) with a &quot;Shipped&quot; column derived from whether a matching shipment
record exists

3. All data must come from shop.db.
Deliver:
- SQL queries used
- Page UI implementation
```

Prompt 3: Place Order Page

```
Create a /place-order page that allows creating a new order for the selected
customer.
Requirements:
1. If no customer selected, redirect to /select-customer.
2. Query products (product_id, product_name, price) and let the user add 1+ line
items:
  - product
  - quantity
```

```
      3. On submit:
        - Insert a row into orders for this customer with order_datetime = current time
        - Insert corresponding rows into order_items
        - Compute and store order_subtotal and order_total in orders (sum of line_total
from order_items)

      4. After placing, redirect to /orders and show a success message.
      Constraints:
      - Use a transaction for inserts.
      - Keep the UI minimal (a table of line items is fine).
      Deliver:
      - SQL inserts
      - Next.js route handlers (server actions or API routes)
      - Any validation rules
```

Prompt 4: Order History Page

```
      Create a /orders page that shows order history for the selected customer.
      Requirements:
      1. If no customer selected, redirect to /select-customer.
      2. Render a table of the customer's orders:
        - order_id, order_datetime, order_total, and a &quot;Shipped&quot; indicator (LEFT
JOIN shipments)

      3. Clicking an order shows /orders/[order_id] with line items:
        - product_name, quantity, unit_price, line_total

      4. Keep it clean and readable.
      Deliver:
      - The two pages
      - SQL queries
```

Prompt 5: Warehouse Priority Queue Page

```
      Create /warehouse/priority page that shows the &quot;Late Delivery Priority
Queue&quot;.
      Use this SQL query exactly (adjust table/column names only if they differ in
shop.db):
      SELECT
        o.order_id,
        o.order_datetime,
        o.order_total,
        c.customer_id,
        c.full_name AS customer_name,
        p.late_delivery_probability,
        p.predicted_late_delivery,
        p.prediction_timestamp

      FROM orders o
      JOIN customers c ON c.customer_id = o.customer_id
      JOIN order_predictions p ON p.order_id = o.order_id
      LEFT JOIN shipments s ON s.order_id = o.order_id
```

```
WHERE s.shipment_id IS NULL
ORDER BY p.late_delivery_probability DESC, o.order_datetime ASC
LIMIT 50;
Requirements:
- Render the results in a table.
- Add a short explanation paragraph describing why this queue exists.
Deliver:
- Page code
```

Prompt 6: Run Scoring Button (Triggers Python Inference Job)

To keep the application simple, the web app will not run ML code. Instead, it triggers a Python inference script that writes predictions into *order_predictions*. The app then reloads the priority queue.

```
Add a /scoring page with a &quot;Run Scoring&quot; button.
Behavior:
1. When clicked, the server runs:
  python jobs/run_inference.py

2. The Python script writes predictions into order_predictions keyed by order_id.
3. The UI shows:
  - Success/failure status
  - How many orders were scored (parse stdout if available)
  - Timestamp

Constraints:
- Provide safe execution: timeouts and capture stdout/stderr.
- The app should not crash if Python fails; show an error message.
- Do not require Docker.
Deliver:
- Next.js route/handler for triggering scoring
- Implementation details for running Python from Node
- Any UI components needed
```

Prompt 7: Polishing and Testing Checklist

```
Polish the app for student usability and add a testing checklist.
Tasks:
1. Add a banner showing which customer is currently selected.
2. Add basic form validation on /place-order.
3. Add error handling for missing DB, missing tables, or empty results.
4. Provide a manual QA checklist:
  - Select customer
  - Place order
  - View orders
  - Run scoring
  - View priority queue with the new order appearing (after scoring)
```

```
Deliver:
- Final code changes
- A README.md with setup and run steps
```

Alternative Stack Prompts

If you prefer Python or C#, you can use the prompts below instead. These prompts generate the same app features but with different frameworks.

Option B: Python (FastAPI) Full-App Prompt

```
Build a complete student web app using Python FastAPI, Jinja2 templates, and SQLite
shop.db (at project root).
No authentication: users select an existing customer to &quot;act as&quot;.
Pages:
- GET /select-customer: list/search customers and store customer_id in a cookie
- GET /dashboard: summary stats for selected customer
- GET/POST /place-order: select products + quantities and insert orders +
order_items
- GET /orders: order history
- GET /orders/{order_id}: order details with line items
- GET /warehouse/priority: priority queue table using order_predictions
- POST /scoring/run: runs python jobs/run_inference.py and then redirects to
/warehouse/priority
Constraints:
- Use sqlite3 (no ORM).
- Use transactions for writes.
- Provide minimal CSS.
- Include a README with setup and run instructions (uvicorn).
Deliver all code files and commands.
```

Option C: ASP.NET/C# Full-App Prompt

```
Build a complete student web app using ASP.NET Core and SQLite shop.db (at project
root).
No authentication: users select an existing customer to &quot;act as&quot; and store
customer_id in a cookie.
Pages/Endpoints:
- /select-customer (GET + POST): choose customer
- /dashboard (GET): customer summary + recent orders
- /place-order (GET + POST): create an order and order_items using a DB transaction
- /orders (GET): order history
- /orders/{orderId} (GET): order detail with line items
- /warehouse/priority (GET): late delivery priority queue (join
orders/customers/order_predictions)
- /scoring/run (POST): execute python jobs/run_inference.py and return status
Constraints:
- Use Microsoft.Data.Sqlite (no EF required).
- Render simple HTML (Razor Pages or MVC ok).
- Provide commands to run (dotnet run) and setup instructions.
```

```
Deliver all code files, NuGet packages, and commands.
```

Key Idea

This app is intentionally simple, but it demonstrates a complete end-to-end pattern: operational data → analytics pipeline → trained model file → automated scoring → operational workflow improvement.

17.9Practice

In this chapter, you built and deployed an end-to-end machine learning pipeline to predict *late delivery* and integrate those predictions directly into an application workflow.

In this practice section, you will extend that pipeline to support an additional predictive task. You may either build a second pipeline from scratch or augment your existing pipeline to handle multiple targets.

Choose a Prediction Target

Select *one* of the following targets to model:

- *is_fraud*: a binary classification task focused on identifying potentially fraudulent orders.
- *risk_score*: a regression task that estimates overall order risk on a continuous scale.

Both targets already exist in the database and were generated using meaningful relationships. Your goal is not to discover a “perfect” model, but to practice designing a clean, deployable pipeline.

Pipeline Design Options

You may approach this practice in one of two ways:

- Build a *separate pipeline* with its own ETL script, training script, and model artifact.
- Extend your existing pipeline to support *multiple targets* using shared feature engineering and separate models.

Both approaches are valid. Choose the one that best matches your comfort level and time constraints.

Required Components

Your solution must include the following elements:

- An ETL step that produces a modeling-ready table in the analytical database.
- Explicit feature selection and a clearly defined target.
- A scikit-learn *Pipeline* that combines preprocessing and modeling.
- Train/test evaluation with appropriate metrics (classification or regression).
- Saved model artifact plus metadata and metrics files.
- Predictions written back to the operational database in a new table keyed by *order_id*.

To avoid naming confusion, use a table name such as *order_predictions_fraud* or *order_predictions_risk*, and include a prediction timestamp column.

Evaluation Guidance

For *is_fraud*, accuracy can be misleading, so include at least precision, recall, and F1 (or PR AUC if you know how). For *risk_score*, include at least MAE (and optionally RMSE).

Application Integration

Decide how your predictions would be used by the application:

- *Fraud prediction*: flag orders for manual review, delayed fulfillment, or additional verification.
- *Risk score*: sort or filter orders by risk level, or combine risk with delivery priority.

You do not need to fully implement the UI changes, but you should be able to explain how the predictions would change system behavior.

AI-Assisted Development

You are encouraged to use Cursor or Claude Code to generate portions of your pipeline. However, you remain responsible for:

- Defining the target and features clearly.
- Ensuring training and inference use identical preprocessing.
- Validating outputs and checking for leakage or overfitting.

Reflection Questions

After completing the exercise, answer the following:

- Which parts of the pipeline were reusable across targets?
- Where did target-specific logic belong?
- Would you deploy this model in a real system? Why or why not?

This practice reinforces a core deployment lesson: scalable ML systems are built from reusable pipelines, not one-off notebooks.

17.10Assignment

Complete the assignment below: